FE 515: Introduction to R

Sean O'Leary

September 03, 2026

Why R?

R is a programming language designed for **data analysis, statistics, and visualization**.

This makes it particularly useful in finance, where we frequently work with:

- Asset prices and returns
- Interest rates
- Portfolio data
- Financial time series
- Regression models
- Simulations
- Optimization problems

Throughout FE 515, we will use R to move from raw financial data to useful analysis.

Think of the workflow:

Data → Store → Manipulate → Analyze

R as a Calculator

At its simplest, R can be used like a calculator.

```
2 + 2
```

```
## [1] 4
```

```
100 * 1.05
```

```
## [1] 105
```

```
100 / 4
```

```
## [1] 25
```

```r
10^2
```

```
## [1] 100
```

The basic arithmetic operators are:

Addition | `+` |
Subtraction | `-` |
Multiplication | `*` |
Division | `/` |
Exponentiation | `^` |

Objects and Assignment

Usually, we want to **store** values so that we can use them later.

In R, the standard assignment operator is `<-`.

```r
price <- 100
shares <- 25
```

We can now use these objects in calculations.

```r
value <- price * shares

value
```

```
## [1] 2500
```

Here:

- `price` stores the value 100
- `shares` stores the value 25
- `value` stores the result of the calculation

Object names should be descriptive.

Vectors

Finance rarely involves only one observation.

We often have a sequence of prices, returns, yields, or other measurements.

In R, a collection of values can be stored in a **vector**.

We create a vector using `c()`, which stands for **combine**.

```r
prices <- c(100, 102, 101, 105, 108)

prices
```

```
## [1] 100 102 101 105 108
```

One of R's most useful features is that calculations can be applied to an entire vector at once.

```r
prices * 2
```

```
## [1] 200 204 202 210 216
```

```r
prices + 1
```

```
## [1] 101 103 102 106 109
```

```r
log(prices)
```

```
## [1] 4.605170 4.624973 4.615121 4.653960 4.682131
```

This is called **vectorization**.

Instead of manually repeating the same calculation for every observation, R performs the operation on the entire vector.

This becomes extremely useful when working with hundreds or thousands of financial observations.

Selecting Values from a Vector

Individual observations can be selected using square brackets.

```r
prices[1]
```

```
## [1] 100
```

```r
prices[3]
```

```
## [1] 101
```

R starts counting at **1**.

We can also select several observations at once.

```r
prices[2:4]
```

```
## [1] 102 101 105
```

The expression `2:4` creates the sequence:

```r
2:4
```

```
## [1] 2 3 4
```

Functions

R contains many built-in functions.

The general structure is:

function_name(argument)

For example:

```r
mean(prices)
```

```
## [1] 103.2
```

```r
min(prices)
```

```
## [1] 100
```

```r
max(prices)
```

```
## [1] 108
```

```r
length(prices)
```

```
## [1] 5
```

We can also calculate the standard deviation.

```r
sd(prices)
```

```
## [1] 3.271085
```

Later in the course, we will use functions for statistical modeling, simulation, optimization, and financial analysis.

If you want to learn more about a function, R includes built-in documentation.

```r
?mean
```

You do **not** need to memorize every R function.

Knowing how to find and use functions is much more important.

Data Frames

Real financial datasets usually contain several variables.

For example, suppose we have several stocks and their prices.

```
stocks <- data.frame(
  ticker = c("AAPL", "MSFT", "JPM"),
  price = c(230, 415, 295)
)

stocks
```

```
##   ticker price
## 1   AAPL   230
## 2   MSFT   415
## 3    JPM   295
```

A **data frame** is similar to a spreadsheet:

- Rows represent observations
- Columns represent variables

We can select a column using `$`.

```
stocks$price
```

```
## [1] 230 415 295
```

Because this column is itself a vector, we can immediately analyze it.

```
mean(stocks$price)
```

```
## [1] 313.3333
```

This idea will become very important when we begin working with real financial datasets.

Our First Financial Calculation

Suppose the following values are daily closing prices for an asset:

```
prices <- c(100, 102, 101, 105, 108)

prices
```

```
## [1] 100 102 101 105 108
```

In finance, we usually care more about **returns** than price levels.

One common measure is the continuously compounded, or log, return:

$$r_t = \log(P_t) - \log(P_{t-1})$$

R has a convenient function called `diff()` that calculates the difference between consecutive observations.

```
diff(c(1, 2, 4, 7, 11, 16))
```

```
## [1] 1 2 3 4 5
```

Therefore, we can calculate log returns using:

```
returns <- diff(log(prices))

returns
```

```
## [1]  0.019802627 -0.009852296  0.038839833  0.028170877
```

Notice that five prices produce four returns.

We can now immediately summarize the returns.

```
mean(returns)
```

```
## [1] 0.01924026
```

```
sd(returns)
```

```
## [1] 0.02090129
```

With only a few lines of R, we have moved from:

$$\text{Prices} \rightarrow \text{Returns} \rightarrow \text{Statistical Analysis}$$

Putting Everything Together

The basic workflow from today's class can be summarized in just a few lines.

```
# Store financial data
prices <- c(100, 102, 101, 105, 108)

# Transform prices into returns
returns <- diff(log(prices))

# Analyze the returns
mean(returns)
```

```
## [1] 0.01924026
```

```r
sd(returns)
```

```
## [1] 0.02090129
```

This simple structure will appear repeatedly throughout FE 515.

Later, we will expand it to work with:

- Real market data
- Descriptive statistics
- Regression
- Monte Carlo simulation
- Root-finding and optimization
- Option pricing

The underlying workflow remains largely the same.

Quick Check

Without running the code first, what do you expect each line to return?

```r
x <- c(10, 12, 14, 16, 18)

length(x)
```

```
## [1] 5
```

```r
x[2]
```

```
## [1] 12
```

```r
mean(x)
```

```
## [1] 14
```

```r
x * 2
```

```
## [1] 20 24 28 32 36
```